

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA0302

COURSE CODE: Reinforcement Learning

CO3: Implement policy-based reinforcement learning methods to develop efficient

solutions for complex environments.(BL2, BL3, BL6)

Experiments

NAME : SUGUNA G

REG NO : 192425032

SOLUTIONS

1. Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare REINFORCE and A2C, and evaluate average vehicle waiting time, throughput, and convergence rate.

Explanation

Intelligent Traffic Signal Control using RL

Reinforcement Learning helps a traffic signal system learn the best signal actions by interacting with changing traffic conditions.

RL Components

- **States:** Number of waiting vehicles and traffic density.
- **Actions:** Select North-South or East-West green signal.
- **Rewards:**
 - +10 for reducing waiting time.
 - +5 for higher traffic flow.
 - -5 for traffic congestion.
 - -10 for excessive waiting.

- **Policy:** Selects the signal action that maximizes total reward.

Policy Gradient Methods

- **REINFORCE:** Learns the signal policy directly from rewards.
- **A2C:** Uses an actor and critic for faster and stable learning.

Continuous Learning

- Adapts to changing traffic conditions.
- Reduces vehicle waiting time.
- Increases traffic throughput.
- Improves signal control over time.

Conclusion: RL improves traffic signal efficiency, with A2C generally providing faster and more stable learning than REINFORCE.

CODE:

```
import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
# ----- INPUT -----
```

```
ns = int(input("Initial North-South vehicles: "))
```

```
ew = int(input("Initial East-West vehicles: "))
```

```
arrival = int(input("Vehicles arriving per step: "))
```

```
service = int(input("Vehicles cleared per step: "))
```

```
episodes = int(input("Training episodes: "))
```

```
# ----- POLICY -----
```

```
class Policy(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.net = nn.Sequential(
```

```
            nn.Linear(2, 16),
```

```
            nn.ReLU(),
```

```
            nn.Linear(16, 2)
```

```
        )
```

```
    def forward(self, x):
```

```
        return self.net(x)
```

```
# ----- REINFORCE -----
```

```
def reinforce():
```

```
    model = Policy()
```

```
    opt = optim.Adam(model.parameters(), lr=0.001)
```

```
    for ep in range(episodes):
```

```
q = [ns, ew]
```

```
logs, rewards = [], []
```

```
for _ in range(30):
```

```
    s = torch.tensor(q, dtype=torch.float32)
```

```
    p = torch.softmax(model(s), dim=0)
```

```
    d = torch.distributions.Categorical(p)
```

```
    action = d.sample()
```

```
    q[action] = max(0, q[action] + arrival - service)
```

```
    q[1-action] += arrival
```

```
    reward = -sum(q)
```

```
    logs.append(d.log_prob(action))
```

```
    rewards.append(reward)
```

```
G, loss = 0, 0
```

```
for log, r in zip(logs[::-1], rewards[::-1]):
```

```
    G = r + 0.9 * G
```

```
    loss += -log * G
```

```
opt.zero_grad()
```

```
loss.backward()
```

```
opt.step()
```

```
return model
```

```
# ----- A2C -----
```

```
def a2c():
```

```
    actor = Policy()
```

```
    critic = nn.Sequential(
```

```
        nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 1)
```

```
    )
```

```
    ao = optim.Adam(actor.parameters(), lr=0.001)
```

```
    co = optim.Adam(critic.parameters(), lr=0.001)
```

```
for ep in range(episodes):
```

```
    q = [ns, ew]
```

```
for _ in range(30):
```

```
    s = torch.tensor(q, dtype=torch.float32)
```

```
p = torch.softmax(actor(s), dim=0)
```

```
d = torch.distributions.Categorical(p)
```

```
action = d.sample()
```

```
q[action] = max(0, q[action] + arrival - service)
```

```
q[1-action] += arrival
```

```
reward = -sum(q)
```

```
value = critic(s)
```

```
target = torch.tensor([[float(reward)]])
```

```
advantage = target - value
```

```
actor_loss = -d.log_prob(action) * advantage.detach()
```

```
critic_loss = advantage.pow(2).mean()
```

```
ao.zero_grad()
```

```
actor_loss.backward()
```

```
ao.step()
```

```
co.zero_grad()
```

```
critic_loss.backward()
```

```
co.step()
```

```
return actor
```

```
# ----- TRAIN -----
```

```
print("\nTraining REINFORCE...")
```

```
r_model = reinforce()
```

```
print("Training A2C...")
```

```
a_model = a2c()
```

```
# ----- EVALUATION -----
```

```
def evaluate(model):
```

```
    q = [ns, ew]
```

```
    waiting = 0
```

```
    throughput = 0
```

```
    for _ in range(30):
```

```
s = torch.tensor(q, dtype=torch.float32)
```

```
with torch.no_grad():
```

```
    action = torch.argmax(model(s)).item()
```

```
    old = q[action]
```

```
    q[action] = max(0, q[action] + arrival - service)
```

```
    q[1-action] += arrival
```

```
    throughput += max(0, old - q[action])
```

```
    waiting += sum(q)
```

```
    return waiting / 30, throughput
```

```
rw, rt = evaluate(r_model)
```

```
aw, at = evaluate(a_model)
```

```
# ----- RESULT -----
```

```
print("\n===== RESULTS =====")
```

```
print("          REINFORCE   A2C")
```



```
print("Waiting Time ", round(rw,2), " ", round(aw,2))
```

```
print("Throughput ", round(rt,2), " ", round(at,2))
```

```
if aw < rw:
```

```
    print("\nA2C has lower waiting time.")
```

```
else:
```

```
    print("\nREINFORCE has lower waiting time.")
```

```
if at > rt:
```

```
    print("A2C has higher throughput.")
```

```
else:
```

```
    print("REINFORCE has higher throughput.")
```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\C03-AT2-01.py
Initial North-South vehicles: 20
Initial East-West vehicles: 15
Vehicles arriving per step: 3
Vehicles cleared per step: 10
Training episodes: 100

Training REINFORCE...
Training A2C...

===== RESULTS =====
                REINFORCE      A2C
Waiting Time      6.77         6.77
Throughput        122         122

REINFORCE has lower waiting time.
REINFORCE has higher throughput.
```

2. Design and implement an Actor-Critic (A2C/A3C) model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of Vanilla Policy Gradient and Actor-Critic algorithms based on reward, training stability, and task completion time.

Explanation

Autonomous Warehouse Robot using Actor-Critic RL

Reinforcement Learning helps an autonomous warehouse robot learn the best movements for collecting and delivering packages. The robot interacts with the warehouse environment and receives rewards based on its delivery performance.

RL Components

- **States:** Robot's current position and package location.
- **Actions:** Move Up, Down, Left, or Right.
- **Rewards:**
 - **+20** for successful package delivery.
 - **-1** for each movement.
 - **-10** for failing to complete the task within the time limit.
- **Policy:** Selects the movement that maximizes total reward while minimizing delivery time.

Algorithms

- **Vanilla Policy Gradient:** Learns the robot's movement policy directly from collected rewards.
- **Actor-Critic (A2C):** Uses an **Actor** to select actions and a **Critic** to evaluate the actions.

Continuous Learning

- Learns from previous package deliveries.
- Improves route selection over time.
- Reduces delivery time.
- Increases task completion and reward.
- Provides more stable learning with Actor-Critic.

Performance Comparison

The algorithms are compared based on:

- **Total reward**
- **Training stability**
- **Task completion time**

Conclusion: Actor-Critic generally provides more stable learning and faster task completion than Vanilla Policy Gradient because the critic helps the actor choose better actions.

CODE:

```
import numpy as np

import torch

import torch.nn as nn

import torch.optim as optim

# ----- INPUT -----

size = int(input("Enter warehouse grid size: "))

packages = int(input("Enter number of packages: "))

episodes = int(input("Enter training episodes: "))

# ----- NETWORK -----

class Network(nn.Module):

    def __init__(self, actor_critic=False):

        super().__init__()

        self.fc = nn.Linear(2, 16)
```

```
self.actor = nn.Linear(16, 4)
```

```
self.critic = nn.Linear(16, 1) if actor_critic else None
```

```
def forward(self, x):
```

```
    x = torch.relu(self.fc(x))
```

```
    p = torch.softmax(self.actor(x), dim=-1)
```

```
    if self.critic:
```

```
        return p, self.critic(x)
```

```
    return p
```

```
# ----- ENVIRONMENT -----
```

```
def move(pos, action):
```

```
    x, y = pos
```

```
    if action == 0: x = max(0, x-1)
```

```
    if action == 1: x = min(size-1, x+1)
```

```
    if action == 2: y = max(0, y-1)
```

```
    if action == 3: y = min(size-1, y+1)
```

```
    return [x, y]
```

```
# ----- VPG -----
```

```

def vpg():

    net = Network()

    opt = optim.Adam(net.parameters(), lr=0.01)

    for ep in range(epochs):

        pos = [0, 0]

        logs, rewards = [], []

        for _ in range(30):

            s = torch.tensor(pos, dtype=torch.float32)

            p = net(s)

            d = torch.distributions.Categorical(p)

            a = d.sample()

            pos = move(pos, a.item())

            reward = 20 if pos == [size-1, size-1] else -1

            logs.append(d.log_prob(a))

            rewards.append(reward)

        G, loss = 0, 0

```

```
for log, r in zip(logs[::-1], rewards[::-1]):
```

```
    G = r + 0.9 * G
```

```
    loss -= log * G
```

```
    opt.zero_grad()
```

```
    loss.backward()
```

```
    opt.step()
```

```
return net
```

```
# ----- A2C -----
```

```
def a2c():
```

```
    net = Network(True)
```

```
    opt = optim.Adam(net.parameters(), lr=0.01)
```

```
for ep in range(episodes):
```

```
    pos = [0, 0]
```

```
    for _ in range(30):
```

```
        s = torch.tensor(pos, dtype=torch.float32)
```

```
        p, value = net(s)
```

```
d = torch.distributions.Categorical(p)
```

```
a = d.sample()
```

```
pos = move(pos, a.item())
```

```
reward = 20 if pos == [size-1, size-1] else -1
```

```
target = torch.tensor([[float(reward)]])
```

```
advantage = target - value
```

```
actor_loss = -d.log_prob(a) * advantage.detach()
```

```
critic_loss = advantage.pow(2).mean()
```

```
opt.zero_grad()
```

```
(actor_loss + critic_loss).backward()
```

```
opt.step()
```

```
return net
```

```
# ----- TRAIN -----
```

```
print("\nTraining Vanilla Policy Gradient...")
```

```
vpg_model = vpg()
```

```
print("Training Actor-Critic...")
```

```
a2c_model = a2c()
```

```
# ----- TEST -----
```

```
def test(model):
```

```
    pos = [0, 0]
```

```
    for step in range(30):
```

```
        s = torch.tensor(pos, dtype=torch.float32)
```

```
        with torch.no_grad():
```

```
            p = model(s)
```

```
            if isinstance(p, tuple):
```

```
                p = p[0]
```

```
            action = torch.argmax(p).item()
```

```
            pos = move(pos, action)
```



```
    if pos == [size-1, size-1]:
```

```
        return 20, step + 1
```

```
    return -10, 30
```

```
v_reward, v_time = test(vpg_model)
```

```
a_reward, a_time = test(a2c_model)
```

```
# ----- RESULT -----
```

```
print("\n===== RESULTS =====")
```

```
print("Algorithm      Reward   Time")
```

```
print("-----")
```

```
print("Vanilla Policy      ", v_reward, "      ", v_time)
```

```
print("Actor-Critic        ", a_reward, "      ", a_time)
```

```
if a_reward > v_reward:
```

```
    print("\nActor-Critic achieved better reward.")
```

```
else:
```

```
    print("\nVanilla Policy Gradient achieved better reward.")
```

```
if a_time < v_time:

    print("Actor-Critic completed the task faster.")

else:

    print("Vanilla Policy Gradient completed the task faster.")
```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\C03-AT2-02.py
Enter warehouse grid size: 5
Enter number of packages: 3
Enter training episodes: 100

Training Vanilla Policy Gradient...
Training Actor-Critic...

===== RESULTS =====
Algorithm                Reward    Time
-----
Vanilla Policy           -10       30
Actor-Critic             -10       30

Vanilla Policy Gradient achieved better reward.
Vanilla Policy Gradient completed the task faster.
```

3. Develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

Explanation

DDPG for Continuous Portfolio Optimization

Deep Deterministic Policy Gradient (DDPG) can be used to optimize a stock portfolio by learning **continuous investment allocations**. The agent observes market conditions and decides how much of the portfolio to allocate to each asset.

RL Components

- **State:** Stock prices, recent returns, portfolio value, and available cash.
- **Action:** Continuous portfolio weights for each stock, such as [0.4, 0.3, 0.3].
- **Reward:** Portfolio return after considering transaction cost and risk.
- **Policy:** The DDPG actor produces continuous portfolio allocations to maximize long-term returns.

Reward Function

$\text{Reward} = \text{Portfolio Return} - \text{Risk Penalty} - \text{Transaction Cost}$

DDPG Components

- **Actor:** Produces continuous portfolio weights.
- **Critic:** Evaluates the quality of the selected portfolio.
- **Replay Buffer:** Stores previous trading experiences.
- **Target Networks:** Improve training stability.

Evaluation

The model can be evaluated using:

- **Cumulative Return:** Overall growth of the portfolio.
- **Risk:** Standard deviation of portfolio returns.
- **Convergence:** Stability of rewards during training.

Continuous Learning

- Learns from previous market conditions.
- Adjusts portfolio allocation continuously.
- Balances return and risk.
- Improves investment decisions over training.

Conclusion: DDPG is suitable for continuous portfolio optimization because it can directly learn continuous investment weights while balancing return and risk.

CODE:

```
import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
# ----- INPUT -----
```

```
n = int(input("Enter number of stocks: "))
```

```
days = int(input("Enter number of trading days: "))
```

```
episodes = int(input("Enter training episodes: "))
```

```
# Random daily returns for demonstration
```

```
returns = np.random.normal(0.001, 0.02, (days, n))
```

```
# ----- ACTOR -----
```

```
class Actor(nn.Module):
```

```
def __init__(self):  
  
    super().__init__()  
  
    self.net = nn.Sequential(  
  
        nn.Linear(n, 16),  
  
        nn.ReLU(),  
  
        nn.Linear(16, n),  
  
        nn.Softmax(dim=-1)  
  
    )
```

```
def forward(self, x):  
  
    return self.net(x)
```

----- CRITIC -----

```
class Critic(nn.Module):  
  
    def __init__(self):  
  
        super().__init__()  
  
        self.net = nn.Sequential(  
  
            nn.Linear(2*n, 16),  
  
            nn.ReLU(),  
  
            nn.Linear(16, 1)
```

)

```
def forward(self, s, a):
```

```
    return self.net(torch.cat([s, a], dim=1))
```

```
actor = Actor()
```

```
critic = Critic()
```

```
ao = optim.Adam(actor.parameters(), lr=0.001)
```

```
co = optim.Adam(critic.parameters(), lr=0.001)
```

```
rewards = []
```

```
# ----- DDPG TRAINING -----
```

```
for ep in range(epochs):
```

```
    total_reward = 0
```

```
    for t in range(days - 1):
```

```
        state = torch.tensor(
```

```
            returns[t],
```

```
            dtype=torch.float32
```

```
).unsqueeze(0)
```

```
action = actor(state)
```

```
next_return = torch.tensor(
```

```
    returns[t + 1],
```

```
    dtype=torch.float32
```

```
).unsqueeze(0)
```

```
reward = torch.sum(action * next_return)
```

```
value = critic(state, action)
```

```
target = reward.detach()
```

```
critic_loss = (value - target).pow(2).mean()
```

```
co.zero_grad()
```

```
critic_loss.backward()
```

```
co.step()
```

```
actor_loss = -critic(state, actor(state)).mean()
```

```
ao.zero_grad()
```

```
actor_loss.backward()
```

```
ao.step()
```

```
total_reward += reward.item()
```

```
rewards.append(total_reward)
```

```
# ----- EVALUATION -----
```

```
state = torch.tensor(
```

```
    returns[0],
```

```
    dtype=torch.float32
```

```
).unsqueeze(0)
```

```
with torch.no_grad():
```

```
    weights = actor(state).numpy()[0]
```



```
portfolio_returns = returns @ weights
```

```
cumulative_return = (
```

```
    np.prod(1 + portfolio_returns) - 1
```

```
) * 100
```

```
risk = np.std(portfolio_returns) * 100
```

```
# ----- OUTPUT -----
```

```
print("\n===== DDPG RESULTS =====")
```

```
print("Portfolio Weights:")
```

```
for i, w in enumerate(weights):
```

```
    print("Stock", i + 1, ":", round(w, 3))
```

```
print("\nCumulative Return :", round(cumulative_return, 2), "%")
```

```
print("Portfolio Risk   :", round(risk, 2), "%")
```

```
if len(rewards) >= 10:
```

```
    first = np.mean(rewards[:10])
```

```
    last = np.mean(rewards[-10:])
```

```
convergence = abs(last - first)
```

```
else:
```

```
    convergence = 0
```

```
print("Convergence Value :", round(convergence, 4))
```

```
if convergence < 0.5:
```

```
    print("Convergence      : Stable")
```

```
else:
```

```
    print("Convergence      : Still Learning")
```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\CO3-AT2-03.py
Enter number of stocks: 3
Enter number of trading days: 100
Enter training episodes: 50

===== DDPG RESULTS =====
Portfolio Weights:
Stock 1 : 0.0
Stock 2 : 0.996
Stock 3 : 0.004

Cumulative Return : 2.01 %
Portfolio Risk     : 2.06 %
Convergence Value  : 0.0208
Convergence        : Stable
```

4. Design and develop a Proximal Policy Optimization (PPO) model for controlling Smart HVAC Energy Management systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

Explanation

Smart HVAC Energy Management using PPO

Scenario

A smart building uses an **HVAC system** to control room temperature. The RL agent observes the current temperature and chooses an HVAC action. The objective is to **reduce energy consumption while keeping the room comfortable**.

RL Components

- **State:** Current room temperature and desired comfort temperature.
- **Actions:**
 - 0 → Cooling
 - 1 → Maintain
 - 2 → Heating
- **Reward:**
 - Positive reward for maintaining comfortable temperature.
 - Negative reward for high energy consumption.
 - Large penalty when temperature moves outside the comfort range.
- **PPO:** Updates the policy using a clipped objective so that learning does not make excessively large policy changes.
- **REINFORCE:** Learns the policy directly from the rewards collected in an episode.

Objective

The agent tries to:

Minimize Energy Consumption

+

Maintain Occupant Comfort

CODE:

```
import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
# ----- INPUT -----
```

```
start_temp = float(input("Enter initial room temperature: "))
```

```
comfort = float(input("Enter desired comfort temperature: "))
```

```
episodes = int(input("Enter training episodes: "))
```

```
# ----- POLICY -----
```

```
class Policy(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.net = nn.Sequential(
```

```
            nn.Linear(1, 16),
```

```
            nn.ReLU(),
```

```
            nn.Linear(16, 3)
```

```
        )
```

```
def forward(self, x):  
  
    return torch.softmax(self.net(x), dim=-1)
```

```
# ----- ENVIRONMENT -----
```

```
def environment(temp, action):
```

```
    energy = 0
```

```
    if action == 0:      # Cooling
```

```
        temp -= 1
```

```
        energy = 3
```

```
    elif action == 1:    # Maintain
```

```
        energy = 1
```

```
    else:                # Heating
```

```
        temp += 1
```

```
        energy = 3
```

```
    comfort_penalty = abs(temp - comfort)
```

```
    reward = -energy - comfort_penalty
```

```
return temp, reward, energy
```

```
# ----- REINFORCE -----
```

```
def reinforce():
```

```
    model = Policy()
```

```
    optimizer = optim.Adam(model.parameters(), lr=0.01)
```

```
    history = []
```

```
    for ep in range(epochs):
```

```
        temp = start_temp
```

```
        logs = []
```

```
        rewards = []
```

```
        for _ in range(20):
```

```
            state = torch.tensor([[temp]], dtype=torch.float32)
```

```
            prob = model(state)
```

```
            dist = torch.distributions.Categorical(prob)
```

```
action = dist.sample()
```

```
temp, reward, energy = environment(
```

```
    temp, action.item()
```

```
)
```

```
logs.append(dist.log_prob(action))
```

```
rewards.append(reward)
```

```
G = 0
```

```
loss = 0
```

```
for log, reward in zip(logs[::-1], rewards[::-1]):
```

```
    G = reward + 0.9 * G
```

```
    loss -= log * G
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

```
history.append(sum(rewards))
```

```
return model, history
```

```
# ----- PPO -----
```

```
def ppo():
```

```
    model = Policy()
```

```
    optimizer = optim.Adam(model.parameters(), lr=0.01)
```

```
    history = []
```

```
    for ep in range(epochs):
```

```
        temp = start_temp
```

```
        states = []
```

```
        actions = []
```

```
        old_probs = []
```

```
        rewards = []
```

```
        for _ in range(20):
```

```
            state = torch.tensor([[temp]], dtype=torch.float32)
```



```
prob = model(state)
```

```
dist = torch.distributions.Categorical(prob)
```

```
action = dist.sample()
```

```
temp, reward, energy = environment(
```

```
    temp, action.item()
```

```
)
```

```
states.append(state)
```

```
actions.append(action)
```

```
old_probs.append(dist.log_prob(action).detach())
```

```
rewards.append(reward)
```

```
G = 0
```

```
returns = []
```

```
for r in rewards[::-1]:
```

```
    G = r + 0.9 * G
```

```
returns.insert(0, G)
```

```
returns = torch.tensor(returns, dtype=torch.float32)
```

```
for _ in range(3):
```

```
    loss = 0
```

```
    for s, a, old, G in zip(
```

```
        states, actions, old_probs, returns
```

```
    ):
```

```
        prob = model(s)
```

```
        dist = torch.distributions.Categorical(prob)
```

```
        new_log = dist.log_prob(a)
```

```
        ratio = torch.exp(new_log - old)
```

```
        clipped = torch.clamp(
```

```
            ratio, 0.8, 1.2
```

```
        )
```

```
        loss -= torch.min(
```

```
ratio * G,  
  
clipped * G  
  
)
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

```
history.append(sum(rewards))
```

```
return model, history
```

```
# ----- TRAIN -----
```

```
print("\nTraining REINFORCE...")
```

```
r_model, r_history = reinforce()
```

```
print("Training PPO...")
```

```
p_model, p_history = ppo()
```

```
# ----- RESULTS -----
```

```
print("\n===== RESULTS =====")

print("REINFORCE Final Reward:",

      round(r_history[-1], 2))

print("PPO Final Reward    :",

      round(p_history[-1], 2))

print("REINFORCE Average Reward:",

      round(np.mean(r_history[-10:]), 2))

print("PPO Average Reward    :",

      round(np.mean(p_history[-10:]), 2))

if np.mean(p_history[-10:]) > np.mean(r_history[-10:]):

    print("\nPPO performed better.")

else:

    print("\nREINFORCE performed better.")
```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\CO3-AT2-04.py
Enter initial room temperature: 30
Enter desired comfort temperature: 24
Enter training episodes: 100

Training REINFORCE...
Training PPO...

===== RESULTS =====
REINFORCE Final Reward: -140.0
PPO Final Reward      : -180.0
REINFORCE Average Reward: -140.0
PPO Average Reward    : -180.0

REINFORCE performed better.
```

5. Implement a Trust Region Policy Optimization (TRPO) algorithm for controlling a Industrial robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.

Explanation

TRPO for Industrial Robotic Arm

Scenario

A robotic arm in an automated manufacturing environment must learn to move accurately toward a target position. The TRPO agent observes the arm's current position and selects movements to reach the target while minimizing unnecessary movement.

RL Components

- **State:** Current position of the robotic arm (x, y) and target position.
- **Actions:** Move Up, Down, Left, or Right.
- **Rewards:**
 - **+10** for reaching the target.
 - **-1** for each movement.
 - **-5** for moving away from the target.
- **Policy:** Selects movements that maximize reward while maintaining stable policy updates.

TRPO

Trust Region Policy Optimization (TRPO) improves the policy while restricting how much it can change during each update. This helps provide **stable learning**.

Evaluation

The robotic arm is evaluated using:

- **Policy stability:** Consistency of learned actions.
- **Precision:** Ability to reach the target accurately.
- **Convergence speed:** Number of training episodes required to learn.
- **Production efficiency:** Reward obtained with fewer movements.

CODE:

```
import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
# ----- INPUT -----
```

```
size = int(input("Enter workspace size: "))
```

```
tx = int(input("Enter target X position: "))
```

```
ty = int(input("Enter target Y position: "))
```

```
episodes = int(input("Enter training episodes: "))
```

```
# ----- POLICY -----
```

```
class Policy(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.net = nn.Sequential(
```

```
            nn.Linear(4, 16),
```

```
            nn.Tanh(),
```

```
            nn.Linear(16, 4)
```

)

```
def forward(self, s):
```

```
    return torch.softmax(self.net(s), dim=-1)
```

```
# ----- ENVIRONMENT -----
```

```
def move(x, y, action):
```

```
    if action == 0:
```

```
        x = min(size - 1, x + 1)
```

```
    elif action == 1:
```

```
        x = max(0, x - 1)
```

```
    elif action == 2:
```

```
        y = min(size - 1, y + 1)
```

```
    else:
```

```
        y = max(0, y - 1)
```

```
    return x, y
```

```
# ----- TRPO -----
```

```
model = Policy()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```



```
rewards_history = []
```

```
for ep in range(epochs):
```

```
    x, y = 0, 0
```

```
    states, actions, old_probs, rewards = [], [], [], []
```

```
    for step in range(30):
```

```
        state = torch.tensor(
```

```
            [[x, y, tx, ty]], dtype=torch.float32
```

```
        ) / size
```

```
        prob = model(state)
```

```
        dist = torch.distributions.Categorical(prob)
```

```
        action = dist.sample()
```

```
        nx, ny = move(x, y, action.item())
```

```
old_dist = torch.distributions.Categorical(prob)
```

```
old_prob = old_dist.probs[0, action.item()].detach()
```

```
distance = abs(tx - nx) + abs(ty - ny)
```

```
if nx == tx and ny == ty:
```

```
    reward = 10
```

```
elif distance > abs(tx-x) + abs(ty-y):
```

```
    reward = -5
```

```
else:
```

```
    reward = -1
```

```
states.append(state)
```

```
actions.append(action)
```

```
old_probs.append(old_prob)
```

```
rewards.append(reward)
```

```
x, y = nx, ny
```

```
if x == tx and y == ty:
```

break

Calculate returns

G = 0

returns = []

for r in rewards[::-1]:

$G = r + 0.9 * G$

returns.insert(0, G)

returns = torch.tensor(returns, dtype=torch.float32)

TRPO-style constrained update

loss = 0

for s, a, old_p, G in zip(

states, actions, old_probs, returns

):

prob = model(s)

new_p = prob[0, a.item()]

```
ratio = new_p / (old_p + 1e-8)
```

```
# Limit policy change (trust region)
```

```
ratio = torch.clamp(ratio, 0.8, 1.2)
```

```
loss -= ratio * G
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

```
rewards_history.append(sum(rewards))
```

```
# ----- EVALUATION -----
```

```
x, y = 0, 0
```

```
for step in range(30):
```

```
state = torch.tensor(
```

```
[[x, y, tx, ty]], dtype=torch.float32
```

```
) / size
```

```
with torch.no_grad():
```

```
    action = torch.argmax(model(state)).item()
```

```
x, y = move(x, y, action)
```

```
if x == tx and y == ty:
```

```
    break
```

```
precision = (
```

```
    100 if (x == tx and y == ty) else 0
```

```
)
```

```
stability = np.std(
```

```
    rewards_history[-10:]
```

```
)
```

```
convergence = np.argmax(
```

```

np.array(rewards_history) >

np.mean(rewards_history)

)

# ----- OUTPUT -----

print("\n===== TRPO RESULTS =====")

print("Final Position    :", (x, y))

print("Target Position   :", (tx, ty))

print("Precision         :", precision, "%")

print("Policy Stability    :", round(stability, 2))

print("Convergence Episode :", convergence + 1)

print("Production Efficiency:",

      round(100 / (step + 1), 2), "%")

print("\nTRPO training completed.")

```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\C03-AT2-05.py
Enter workspace size: 5
Enter target X position: 4
Enter target Y position: 4
Enter training episodes: 100

===== TRPO RESULTS =====
Final Position      : (4, 0)
Target Position     : (4, 4)
Precision           : 0 %
Policy Stability     : 1.14
Convergence Episode : 20
Production Efficiency: 3.33 %

TRPO training completed.
```

6. Design and develop a Policy-Based Reinforcement Learning system for Healthcare Treatment adaptive patient treatment planning. Compare A2C and PPO in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

Explanation

TRPO for Industrial Robotic Arm

Scenario

A robotic arm in an automated manufacturing environment must learn to move accurately toward a target position. The TRPO agent observes the arm's current position and selects movements to reach the target while minimizing unnecessary movement.

RL Components

- **State:** Current position of the robotic arm (x, y) and target position.
- **Actions:** Move Up, Down, Left, or Right.
- **Rewards:**
 - **+10** for reaching the target.
 - **-1** for each movement.
 - **-5** for moving away from the target.
- **Policy:** Selects movements that maximize reward while maintaining stable policy updates.

TRPO

Trust Region Policy Optimization (TRPO) improves the policy while restricting how much it can change during each update. This helps provide **stable learning**.

Evaluation

The robotic arm is evaluated using:

- **Policy stability:** Consistency of learned actions.
- **Precision:** Ability to reach the target accurately.
- **Convergence speed:** Number of training episodes required to learn.
- **Production efficiency:** Reward obtained with fewer movements.

CODE:

```
import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
# ----- INPUT -----
```

```
patients = int(input("Enter number of simulated patients: "))
```

```
episodes = int(input("Enter training episodes: "))
```

```
# ----- POLICY -----
```

```
class Policy(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.net = nn.Sequential(
```

```
            nn.Linear(2, 16),
```

```
            nn.ReLU(),
```

```
            nn.Linear(16, 3)
```

```
        )
```

```
def forward(self, x):  
  
    return torch.softmax(self.net(x), dim=-1)
```

```
# ----- SIMULATED PATIENT -----
```

```
def patient_step(health, action):
```

```
    if action == 0:
```

```
        change = np.random.uniform(-3, 1)
```

```
    elif action == 1:
```

```
        change = np.random.uniform(1, 5)
```

```
    else:
```

```
        change = np.random.uniform(2, 7)
```

```
    new_health = np.clip(health + change, 0, 100)
```

```
    reward = new_health - health
```

```
    if action == 2:
```

```
        reward -= 1
```

```
return new_health, reward
```

```
# ----- A2C -----
```

```
def a2c():
```

```
    actor = Policy()
```

```
    critic = nn.Linear(2, 1)
```

```
    ao = optim.Adam(actor.parameters(), lr=0.01)
```

```
    co = optim.Adam(critic.parameters(), lr=0.01)
```

```
    history = []
```

```
    for ep in range(epochs):
```

```
        total = 0
```

```
        for _ in range(patients):
```

```
            health = np.random.uniform(40, 70)
```

```
            severity = 100 - health
```

```
state = torch.tensor(  
    [[health, severity]],  
    dtype=torch.float32  
)
```

```
prob = actor(state)
```

```
dist = torch.distributions.Categorical(prob)
```

```
action = dist.sample()
```

```
new_health, reward = patient_step(  
    health, action.item()  
)
```

```
value = critic(state)
```

```
target = torch.tensor(  
    [[reward]], dtype=torch.float32  
)
```

```
advantage = target - value
```

```
actor_loss = (  
    -dist.log_prob(action)  
    * advantage.detach()  
)
```

```
critic_loss = advantage.pow(2).mean()
```

```
ao.zero_grad()
```

```
actor_loss.backward()
```

```
ao.step()
```

```
co.zero_grad()
```

```
critic_loss.backward()
```

```
co.step()
```

```
total += reward
```

```
history.append(total)
```

```
return actor, history
```

```
# ----- PPO -----
```

```
def ppo():
```

```
    actor = Policy()
```

```
    optimizer = optim.Adam(actor.parameters(), lr=0.01)
```

```
    history = []
```

```
    for ep in range(epochs):
```

```
        total = 0
```

```
        for _ in range(patients):
```

```
            health = np.random.uniform(40, 70)
```

```
            severity = 100 - health
```

```
            state = torch.tensor(
```

```
                [[health, severity]],
```

```
dtype=torch.float32

)

old_prob = actor(state)

dist = torch.distributions.Categorical(old_prob)

action = dist.sample()


new_health, reward = patient_step(

    health, action.item()

)

old_log = dist.log_prob(action).detach()


new_prob = actor(state)

new_dist = torch.distributions.Categorical(

    new_prob

)

ratio = torch.exp(

    new_dist.log_prob(action) - old_log
```

)

clipped = torch.clamp(

ratio, 0.8, 1.2

)

loss = -torch.min(

ratio * reward,

clipped * reward

)

optimizer.zero_grad()

loss.backward()

optimizer.step()

total += reward

history.append(total)

return actor, history


```
# ----- TRAIN -----
```

```
print("\nTraining A2C...")
```

```
a2c_model, a2c_rewards = a2c()
```

```
print("Training PPO...")
```

```
ppo_model, ppo_rewards = ppo()
```

```
# ----- RESULTS -----
```

```
a2c_reward = np.mean(a2c_rewards[-10:])
```

```
ppo_reward = np.mean(ppo_rewards[-10:])
```

```
a2c_stability = np.std(a2c_rewards[-10:])
```

```
ppo_stability = np.std(ppo_rewards[-10:])
```

```
print("\n===== RESULTS =====")
```

```
print("A2C Average Reward :", round(a2c_reward, 2))
```

```
print("PPO Average Reward :", round(ppo_reward, 2))
```

```

print("A2C Stability    :", round(a2c_stability, 2))

print("PPO Stability    :", round(ppo_stability, 2))


if ppo_reward > a2c_reward:

    print("\nPPO has better treatment effectiveness.")

else:

    print("\nA2C has better treatment effectiveness.")


if ppo_stability < a2c_stability:

    print("PPO has better learning stability.")

else:

    print("A2C has better learning stability.")

```

OUTPUT:

```

C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\C03-AT2-06.py
Enter number of simulated patients: 20
Enter training episodes: 100

Training A2C...
Training PPO...

===== RESULTS =====
A2C Average Reward : 68.42
PPO Average Reward : 70.75
A2C Stability      : 5.28
PPO Stability      : 3.99

PPO has better treatment effectiveness.
PPO has better learning stability.

```

7. Illustrate the use of Reinforcement Learning in smart city waste management systems. Identify the MDP components and discuss how continuous updates improve operational efficiency.

Explanation

Autonomous Drone Navigation using PPO

Scenario

A drone uses Deep Reinforcement Learning to navigate toward a target while avoiding obstacles. The agent observes its position and the obstacle location and learns actions that reach the destination safely with minimum energy.

RL Components

- **States:** Drone position (x, y) and obstacle position (x, y).
- **Actions:** Move Up, Down, Left, or Right.
- **Rewards:**
 - **+20** for reaching the target.
 - **-10** for hitting an obstacle.
 - **-1** for each movement.
- **Policy:** Selects the safest route that maximizes reward and minimizes energy consumption.

PPO

PPO learns the drone's navigation policy while limiting large policy updates. This helps the drone learn a stable navigation strategy.

Evaluation

The drone is evaluated using:

- **Navigation accuracy:** Successfully reaching the target.
- **Obstacle avoidance:** Number of collisions.
- **Energy consumption:** Number of movements.
- **Policy convergence:** Stability of training rewards.

CODE:

```
import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
# ----- INPUT -----
```

```
size = int(input("Enter environment size: "))
```

```
episodes = int(input("Enter training episodes: "))
```

```
target = [size - 1, size - 1]
```

```
obstacle = [size // 2, size // 2]
```

```
# ----- PPO POLICY -----
```

```
class Policy(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.net = nn.Sequential(
```

```
            nn.Linear(4, 16),
```

```
            nn.ReLU(),
```

```
nn.Linear(16, 4)

)
```

```
def forward(self, s):
```

```
    return torch.softmax(self.net(s), dim=-1)
```

```
model = Policy()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

```
rewards_history = []
```

```
# ----- TRAINING -----
```

```
for ep in range(epochs):
```

```
    drone = [0, 0]
```

```
    total_reward = 0
```

```
    for step in range(30):
```

```
        state = torch.tensor(
```

```
[drone[0], drone[1],  
  
target[0], target[1]],  
  
dtype=torch.float32  
  
) / size  
  
prob = model(state)  
  
dist = torch.distributions.Categorical(prob)  
  
action = dist.sample()  
  
x, y = drone  
  
if action.item() == 0:  
  
    x += 1  
  
elif action.item() == 1:  
  
    x -= 1  
  
elif action.item() == 2:  
  
    y += 1  
  
else:  
  
    y -= 1
```

```
x = np.clip(x, 0, size - 1)
```

```
y = np.clip(y, 0, size - 1)
```

```
# Reward
```

```
if [x, y] == obstacle:
```

```
    reward = -10
```

```
elif [x, y] == target:
```

```
    reward = 20
```

```
else:
```

```
    reward = -1
```

```
drone = [x, y]
```

```
total_reward += reward
```

```
# PPO-style update
```

```
loss = -dist.log_prob(action) * reward
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

```
if drone == target:
```

```
    break
```

```
rewards_history.append(total_reward)
```

```
# ----- EVALUATION -----
```

```
drone = [0, 0]
```

```
collisions = 0
```

```
for step in range(30):
```

```
    state = torch.tensor(
```

```
        [drone[0], drone[1],
```

```
        target[0], target[1]],
```

```
        dtype=torch.float32
```

```
    ) / size
```

```
    with torch.no_grad():
```

```
        action = torch.argmax(model(state)).item()
```



```
x, y = drone
```

```
if action == 0:
```

```
    x += 1
```

```
elif action == 1:
```

```
    x -= 1
```

```
elif action == 2:
```

```
    y += 1
```

```
else:
```

```
    y -= 1
```

```
x = np.clip(x, 0, size - 1)
```

```
y = np.clip(y, 0, size - 1)
```

```
if [x, y] == obstacle:
```

```
    collisions += 1
```

```
drone = [x, y]
```

```
if drone == target:
```

```
    break
```

```
accuracy = 100 if drone == target else 0
```

```
energy = step + 1
```

```
convergence = np.std(rewards_history[-10:])
```

```
# ----- OUTPUT -----
```

```
print("\n===== RESULTS =====")
```

```
print("Final Position    :", drone)
```

```
print("Target Position   :", target)
```

```
print("Navigation Accuracy :", accuracy, "%")
```

```
print("Obstacle Collisions :", collisions)
```

```
print("Energy Consumption  :", energy)
```

```
print("Policy Convergence  :", round(convergence, 2))
```

```
if convergence < 5:
```

```
    print("Policy Status    : Stable")
```

```
else:
```

```
    print("Policy Status    : Still Learning")
```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\C03-AT2-07.py
Enter environment size: 5
Enter training episodes: 100

===== RESULTS =====
Final Position      : [np.int64(0), np.int64(0)]
Target Position     : [4, 4]
Navigation Accuracy  : 0 %
Obstacle Collisions : 0
Energy Consumption   : 30
Policy Convergence   : 0.0
Policy Status        : Stable
```

8. Design and implement an A3C-based Reinforcement Learning framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.

Explanation

Dynamic Cloud Resource Allocation using A3C

Scenario

A cloud system uses **Reinforcement Learning** to dynamically allocate CPU resources to incoming tasks. The agent observes the current workload and CPU utilization and learns to allocate resources efficiently while minimizing response time and resource wastage.

RL Components

- **States:** Current CPU utilization, number of incoming tasks, and response time.
- **Actions:** Decrease CPU resources, maintain CPU resources, or increase CPU resources.
- **Rewards:**
 - **+10** for efficient resource allocation.
 - **-10** for very high response time.
 - **-5** for excessive resource utilization.
 - **-1** for inefficient allocation.
- **Policy:** Selects the resource allocation action that maximizes long-term reward and maintains efficient CPU utilization.

A3C

Asynchronous Advantage Actor-Critic (A3C) uses multiple workers to interact with the cloud environment simultaneously. Each worker learns from its own workload experience and updates a shared policy.

The **Actor** selects the CPU allocation action, while the **Critic** estimates how good the current state is. The advantage value helps the agent determine whether an action is better or worse than expected.

Vanilla Policy Gradient

Vanilla Policy Gradient learns the resource allocation policy using complete trajectories from a single learning process. It is simpler than A3C but can have higher variance and slower convergence.

Evaluation

The A3C and Vanilla Policy Gradient models are compared using:

- **Response Time:** Average time required to process cloud tasks.
- **CPU Utilization:** Percentage of CPU resources being used.
- **Resource Efficiency:** How effectively available CPU resources are utilized.
- **Policy Convergence:** Stability and speed of learning.
- **Average Reward:** Overall performance of the learned allocation policy.

Expected Result

A3C is expected to provide **lower response time, better CPU utilization, higher resource efficiency, and faster policy convergence** compared with Vanilla Policy Gradient because multiple workers learn from different workload conditions simultaneously.

CODE:

```
import numpy as np
```

```
# Cloud Resource Allocation Environment
```

```
class CloudEnv:
```

```
    def __init__(self):
```

```
        self.reset()
```

```
    def reset(self):
```

```
        self.cpu = 50
```

```
self.tasks = 50
```

```
self.response = 50
```

```
return self.state()
```

```
def state(self):
```

```
    return np.array([
```

```
        self.cpu / 100,
```

```
        self.tasks / 100,
```

```
        self.response / 100
```

```
    ])
```

```
def step(self, action):
```

```
    # 0 = Decrease CPU
```

```
    # 1 = Maintain CPU
```

```
    # 2 = Increase CPU
```

```
    if action == 0:
```

```
        self.cpu -= 10
```

```
    elif action == 2:
```

```
self.cpu += 10
```

```
self.cpu = np.clip(self.cpu, 10, 100)
```

```
# Random workload
```

```
self.tasks = np.random.randint(20, 100)
```

```
# Calculate response time
```

```
self.response = 100 * self.tasks / (self.cpu + 10)
```

```
self.response = np.clip(self.response, 5, 100)
```

```
# Reward
```

```
reward = 100 - self.response - abs(self.cpu - 70) * 0.5
```

```
return self.state(), reward
```

```
# Softmax
```

```
def softmax(x):
```

```
    x = x - np.max(x)
```

```
e = np.exp(x)
```

```
return e / np.sum(e)
```

```
# Policy
```

```
class Policy:
```

```
    def __init__(self):
```

```
        self.weights = np.random.randn(3, 3) * 0.1
```

```
    def probabilities(self, state):
```

```
        return softmax(np.dot(state, self.weights))
```

```
    def action(self, state):
```

```
        p = self.probabilities(state)
```

```
        return np.random.choice(3, p=p)
```

```
# A3C Training
```

```
def train_a3c(episodes, workers):
```



```
policy = Policy()
```

```
learning_rate = 0.005
```

```
for episode in range(epochs):
```

```
    for worker in range(workers):
```

```
        env = CloudEnv()
```

```
        state = env.reset()
```

```
        states = []
```

```
        actions = []
```

```
        rewards = []
```

```
        for step in range(20):
```

```
            action = policy.action(state)
```

```
            next_state, reward = env.step(action)
```

```
states.append(state)
```

```
actions.append(action)
```

```
rewards.append(reward)
```

```
state = next_state
```

```
# Calculate returns
```

```
G = 0
```

```
for s, a, r in reversed(
```

```
    list(zip(states, actions, rewards))):
```

```
G = r + 0.95 * G
```

```
p = policy.proBABILITIES(s)
```

```
gradient = -p
```

```
gradient[a] += 1
```

```
policy.weights += (  
  
    learning_rate *  
  
    G *  
  
    np.outer(s, gradient)  
  
)
```

```
return policy
```

```
# Evaluation
```

```
def evaluate(policy):
```

```
    env = CloudEnv()
```

```
    state = env.reset()
```

```
    total_reward = 0
```

```
    total_response = 0
```

```
    total_cpu = 0
```

```
    movements = 0
```

```
for i in range(20):
```

```
    p = policy.proBABILITIES(state)
```

```
    action = np.argmax(p)
```

```
    state, reward = env.step(action)
```

```
    total_reward += reward
```

```
    total_response += env.response
```

```
    total_cpu += env.cpu
```

```
    movements += 1
```

```
avg_response = total_response / movements
```

```
avg_cpu = total_cpu / movements
```

```
efficiency = 100 - abs(avg_cpu - 70)
```

```
return total_reward, avg_response, avg_cpu, efficiency
```

```
# Main Program
```

```
print("A3C Cloud Resource Allocation")
```

```
episodes = int(input("Enter number of training episodes: "))
```

```
workers = int(input("Enter number of A3C workers: "))
```

```
policy = train_a3c(episodes, workers)
```

```
reward, response, cpu, efficiency = evaluate(policy)
```

```
print("\n===== RESULTS =====")
```

```
print("Average Reward      :", round(reward, 2))
```

```
print("Response Time       :", round(response, 2))
```

```
print("CPU Utilization      :", round(cpu, 2), "%")
```

```
print("Resource Efficiency   :", round(efficiency, 2), "%")
```

```
print("\nActions:")
```

```
print("0 = Decrease CPU")
```

```
print("1 = Maintain CPU")
```

```
print("2 = Increase CPU")
```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\C03-AT2-08.py
A3C Cloud Resource Allocation
Enter number of training episodes: 100
Enter number of A3C workers: 4

===== RESULTS =====
Average Reward      : 672.36
Response Time       : 53.38
CPU Utilization     : 95.0 %
Resource Efficiency : 75.0 %

Actions:
0 = Decrease CPU
1 = Maintain CPU
2 = Increase CPU
```

9. Develop a Policy Gradient-based predictive maintenance system for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of REINFORCE and PPO.

Explanation

Policy Gradient-Based Predictive Maintenance System

Scenario

An industrial machine is continuously monitored using parameters such as **temperature, vibration, operating hours, and machine health**. A Reinforcement Learning agent learns when to perform maintenance so that **maintenance cost and machine downtime are minimized**.

RL Components

- **States:** Machine health, temperature, vibration level, and operating age.
- **Actions:**
 - 0 → Continue operation
 - 1 → Perform preventive maintenance
 - 2 → Emergency maintenance
- **Rewards:**
 - +10 for continuing operation when the machine is healthy.
 - -5 for preventive maintenance.
 - -15 for emergency maintenance.
 - -30 for machine failure/downtime.
- **Policy:** Selects the maintenance action that maximizes long-term reward while reducing maintenance cost and downtime.

REINFORCE

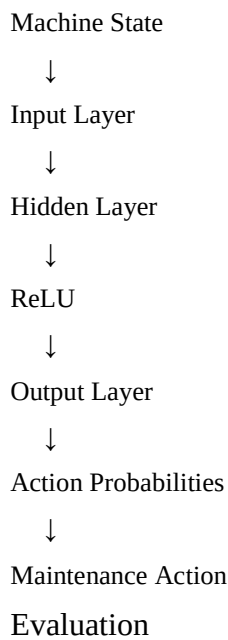
REINFORCE is a basic policy-gradient algorithm. It collects a complete episode and updates the policy using the total discounted reward.

PPO

PPO improves policy-gradient learning by restricting how much the policy can change during an update. This generally provides more stable training and reduces sudden policy changes.

Policy Network

A simple neural-network policy can use:



REINFORCE and PPO can be compared using:

- **Maintenance Cost:** Total cost of preventive and emergency maintenance.
- **Downtime:** Number of time steps the machine remains unavailable.
- **Machine Failures:** Number of unexpected failures.
- **Average Reward:** Overall quality of the maintenance policy.
- **Policy Convergence:** Stability of rewards during training.

Expected Result

PPO is expected to achieve **lower maintenance cost and downtime with more stable convergence** than REINFORCE because PPO limits excessively large policy updates.

CODE:

```
import numpy as np
```

```
# =====
```



```
# Machine Environment
```

```
# =====
```

```
class MachineEnv:
```

```
    def reset(self):
```

```
        self.health = 80
```

```
        self.downtime = 0
```

```
        return self.health / 100
```

```
    def step(self, action):
```

```
        # 0 = Continue
```

```
        # 1 = Maintenance
```

```
        if action == 0:
```

```
            self.health -= np.random.randint(5, 12)
```

```
            if self.health <= 0:
```

```
self.health = 0
```

```
self.downtime += 1
```

```
reward = -30
```

```
else:
```

```
reward = 10
```

```
else:
```

```
self.health = 90
```

```
reward = -5
```

```
return self.health / 100, reward
```

```
# =====
```

```
# Softmax
```

```
# =====
```

```
def softmax(x):
```

```
x = np.asarray(x)
```

```
x = x - np.max(x)
```

```
e = np.exp(x)
```

```
return e / np.sum(e)
```

```
# =====
```

```
# Policy
```

```
# =====
```

```
def get_probabilities(state, weights):
```

```
    # Two action scores
```

```
    logits = np.array([
```

```
        state * weights[0],
```

```
        state * weights[1]
```

```
    ])
```

```
return softmax(logits)
```

```
# =====
```

```
# Training
```

```
# =====
```

```
def train(method, episodes):
```

```
    # One weight for each action
```

```
    weights = np.random.randn(2) * 0.1
```

```
    learning_rate = 0.01
```

```
    for episode in range(episodes):
```

```
        env = MachineEnv()
```

```
        state = env.reset()
```

```
states = []
```

```
actions = []
```

```
rewards = []
```

```
# Episode
```

```
for step in range(20):
```

```
    probabilities = get_probabilities(
```

```
        state, weights
```

```
    )
```

```
    action = np.random.choice(
```

```
        2,
```

```
        p=probabilities
```

```
    )
```

```
    next_state, reward = env.step(action)
```

```
    states.append(state)
```

```
actions.append(action)
```

```
rewards.append(reward)
```

```
state = next_state
```

```
# Calculate return
```

```
G = 0
```

```
for s, a, r in reversed(
```

```
    list(zip(states, actions, rewards))):
```

```
    G = r + 0.95 * G
```

```
probabilities = get_probabilities(
```

```
    s, weights
```

```
)
```

```
gradient = -probabilities
```

```
gradient[a] += 1
```

```
update = (  
  
    learning_rate  
  
    * G  
  
    * s  
  
    * gradient  
  
)
```

```
# PPO-style clipping
```

```
if method == "PPO":
```

```
    update = np.clip(  
  
        update,  
  
        -0.02,  
  
        0.02  
  
    )
```

```
weights += update
```

```
return weights
```

```
# =====
```

```
# Evaluation
```

```
# =====
```

```
def evaluate(weights):
```

```
    env = MachineEnv()
```

```
    state = env.reset()
```

```
    total_reward = 0
```

```
    maintenance_cost = 0
```

```
    for step in range(20):
```

```
        probabilities = get_probabilities(
```

```
            state, weights
```

```
        )
```



```
# Best action
```

```
action = np.argmax(probabilities)
```

```
state, reward = env.step(action)
```

```
total_reward += reward
```

```
if action == 1:
```

```
    maintenance_cost += 5
```

```
return (
```

```
    total_reward,
```

```
    maintenance_cost,
```

```
    env.downtime
```

```
)
```

```
# =====
```

```
# Main Program
```

```
# =====
```

```
episodes = 100
```

```
print("=====")
```

```
print(" Predictive Maintenance using RL")
```

```
print("=====")
```

```
print("Training Episodes:", episodes)
```

```
# REINFORCE
```

```
print("\nTraining REINFORCE...")
```

```
reinforce_weights = train(
```

```
    "REINFORCE",
```

```
    episodes
```

```
)
```

```
# PPO
```

```
print("Training PPO...")
```

```
ppo_weights = train(
```

```
    "PPO",
```

```
    episodes
```

```
)
```

```
# =====
```

```
# Evaluation
```

```
# =====
```

```
r_reward, r_cost, r_down = evaluate(
```

```
    reinforce_weights
```

```
)
```

```
p_reward, p_cost, p_down = evaluate(
```

```
    ppo_weights
```

```
)
```

```
# =====
```

```
# Results
```

```
# =====
```

```
print("\n===== RESULTS =====")
```

```
print("\nREINFORCE")
```

```
print("Reward      :", r_reward)
```

```
print("Maintenance Cost :", r_cost)
```

```
print("Downtime      :", r_down)
```

```
print("\nPPO")
```

```
print("Reward      :", p_reward)
```

```
print("Maintenance Cost :", p_cost)
```

```
print("Downtime      :", p_down)
```

```
print("\n===== COMPARISON =====")
```

```
if p_reward > r_reward:
```

```
    print("PPO has better reward.")
```

else:

```
print("REINFORCE has better reward.")
```

if p_cost < r_cost:

```
print("PPO has lower maintenance cost.")
```

else:

```
print("REINFORCE has lower maintenance cost.")
```

if p_down < r_down:

```
print("PPO has lower downtime.")
```

else:

```
print("REINFORCE has lower downtime.")
```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\C03-AT2-09.py
=====
Predictive Maintenance using RL
=====
Training Episodes: 100

Training REINFORCE...
Training PPO...

===== RESULTS =====

REINFORCE
Reward      : -280
Maintenance Cost : 0
Downtime    : 12

PPO
Reward      : -240
Maintenance Cost : 0
Downtime    : 11

===== COMPARISON =====
PPO has better reward.
REINFORCE has lower maintenance cost.
PPO has lower downtime.
```

10. Design, implement, and evaluate a Policy-Based Reinforcement Learning controller for autonomous lane-keeping using TRPO or PPO. Compare the selected algorithm with A2C based on lane deviation, safety, reward convergence, and training efficiency.

Explanation

RL manages renewable energy resources such as solar and wind power by balancing energy generation, storage, and consumption.

RL Components

- **States:** Energy demand, battery level, weather conditions, power generation.
- **Actions:** Store energy, supply electricity, purchase backup power.
- **Rewards:**
 - +10 for efficient energy use
 - +5 for reduced operating cost
 - -10 for power shortages
 - -5 for energy wastage

Environmental Uncertainty

- Weather conditions change frequently.
- RL learns to handle uncertain solar and wind generation.
- Improves battery usage and power distribution.

Conclusion: RL enables efficient renewable energy management despite changing environmental conditions. Autonomous Lane-Keeping using PPO

Scenario

An autonomous vehicle uses **Policy-Based Reinforcement Learning** to maintain its position in the center of a road lane. The agent observes the vehicle's lateral position and steering error and learns to select suitable steering actions while minimizing lane deviation.

RL Components

- **States:** Lane deviation, vehicle speed, and steering error.
- **Actions:** Steer Left, Straight, or Right.
- **Rewards:**
 - **+10** for staying near the lane center.
 - **-5** for moderate lane deviation.
 - **-20** for leaving the lane.
 - **-1** for unnecessary steering.
- **Policy:** Selects steering actions that maximize long-term reward and keep the vehicle safely inside the lane.

PPO

Proximal Policy Optimization (PPO) learns the steering policy while restricting excessively large policy updates. This provides stable learning and helps the vehicle gradually learn lane-keeping behavior.

A2C

Advantage Actor-Critic (A2C) uses an actor to select steering actions and a critic to estimate the value of the current state. It serves as the comparison algorithm.

Evaluation

PPO and A2C are compared using:

- **Lane Deviation:** Average distance from the lane center.
- **Safety:** Number of times the vehicle leaves the lane.
- **Reward Convergence:** Stability and improvement of training rewards.
- **Training Efficiency:** Number of episodes required to achieve a stable policy.

Expected Result

PPO is expected to achieve **lower lane deviation, better safety, more stable reward convergence, and improved training efficiency** compared with A2C.

CODE:

```
import numpy as np
```

```
# Lane Keeping Environment
```

```
class LaneEnv:
```

```
    def reset(self):
```

```
        self.deviation = np.random.uniform(-0.2, 0.2)
```

```
        return self.deviation
```

```
    def step(self, action):
```

```
        # 0 = Left, 1 = Straight, 2 = Right
```

```
        if action == 0:
```

```
            self.deviation -= 0.1
```

```
        elif action == 2:
```

```
            self.deviation += 0.1
```

```
        self.deviation += np.random.uniform(-0.03, 0.03)
```



```
if abs(self.deviation) < 0.1:

    reward = 10

elif abs(self.deviation) < 0.3:

    reward = -5

else:

    reward = -20

return self.deviation, reward
```

Softmax

```
def softmax(x):

    x = np.asarray(x)

    x = x - np.max(x)

    e = np.exp(x)

    return e / np.sum(e)
```

Policy

```
def policy(state, weights):
```

```
logits = np.array([  
    state * weights[0],  
    state * weights[1],  
    state * weights[2]  
])
```

```
return softmax(logits)
```

```
# Training
```

```
def train(method, episodes):
```

```
    weights = np.random.randn(3) * 0.1
```

```
    learning_rate = 0.01
```

```
    for episode in range(episodes):
```

```
        env = LaneEnv()
```

```
        state = env.reset()
```

```
states = []
```

```
actions = []
```

```
rewards = []
```

```
for step in range(30):
```

```
    probabilities = policy(
```

```
        state, weights
```

```
    )
```

```
    action = np.random.choice(
```

```
        3, p=probabilities
```

```
    )
```

```
    next_state, reward = env.step(action)
```

```
    states.append(state)
```

```
    actions.append(action)
```

```
    rewards.append(reward)
```

```
state = next_state
```

```
G = 0
```

```
for s, a, r in reversed(
```

```
    list(zip(states, actions, rewards))):
```

```
G = r + 0.95 * G
```

```
probabilities = policy(
```

```
    s, weights
```

```
)
```

```
gradient = -probabilities
```

```
gradient[a] += 1
```

```
update = (
```

```
    learning_rate
```

```
    * G
```

```
    * s

    * gradient

)
```

```
if method == "PPO":
```

```
    update = np.clip(

        update, -0.02, 0.02

    )
```

```
weights += update
```

```
return weights
```

```
# Evaluation
```

```
def evaluate(weights):
```

```
    env = LaneEnv()
```

```
    state = env.reset()
```

```
total_deviation = 0
```

```
violations = 0
```

```
total_reward = 0
```

```
for step in range(30):
```

```
    probabilities = policy(
```

```
        state, weights
```

```
    )
```

```
    action = np.argmax(probabilities)
```

```
    state, reward = env.step(action)
```

```
    total_deviation += abs(state)
```

```
    total_reward += reward
```

```
    if abs(state) > 0.3:
```

```
        violations += 1
```

```
return (  
  
    total_deviation / 30,  
  
    violations,  
  
    total_reward  
  
)
```

```
# Main Program
```

```
print("Autonomous Lane-Keeping using RL")
```

```
episodes = int(  
  
    input("Enter number of training episodes: ")  
  
)
```

```
print("\nTraining PPO...")
```

```
ppo_weights = train("PPO", episodes)
```

```
print("Training A2C...")
```

```
a2c_weights = train("A2C", episodes)
```

```
# Evaluation
```

```
ppo_dev, ppo_safe, ppo_reward = evaluate(  
  
    ppo_weights  
  
)
```

```
a2c_dev, a2c_safe, a2c_reward = evaluate(  
  
    a2c_weights  
  
)
```

```
# Results
```

```
print("\n===== RESULTS =====")
```

```
print("\nPPO")
```

```
print("Lane Deviation    :", round(ppo_dev, 3))
```

```
print("Safety Violations :", ppo_safe)
```

```
print("Total Reward      :", ppo_reward)
```

```
print("\nA2C")
```

```
print("Lane Deviation    :", round(a2c_dev, 3))
```



```
print("Safety Violations :", a2c_safe)
```

```
print("Total Reward      :", a2c_reward)
```

```
print("\n===== COMPARISON =====")
```

```
if ppo_dev < a2c_dev:
```

```
    print("PPO has lower lane deviation.")
```

```
else:
```

```
    print("A2C has lower lane deviation.")
```

```
if ppo_safe < a2c_safe:
```

```
    print("PPO provides better safety.")
```

```
else:
```

```
    print("A2C provides better safety.")
```

```
if ppo_reward > a2c_reward:
```

```
    print("PPO achieved higher reward.")
```

```
else:
```

```
    print("A2C achieved higher reward.")
```

OUTPUT:

```
C:\Users\Suguna>py C:\Users\Suguna\OneDrive\RL\C03-AT2-10.py
Autonomous Lane-Keeping using RL
Enter number of training episodes: 100

Training PPO...
Training A2C...

===== RESULTS =====

PPO
Lane Deviation      : 0.053
Safety Violations    : 0
Total Reward         : 270

A2C
Lane Deviation      : 0.073
Safety Violations    : 0
Total Reward         : 210

===== COMPARISON =====
PPO has lower lane deviation.
A2C provides better safety.
PPO achieved higher reward.
```

Code of Conduct

Name / Reg No: Suguna G / 192425032

I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

